CHAPTER 32

Soundness, unsafe and Semantic Types

In Chapter 9 we proved that a small language is type safe: a well-typed program never gets stuck. It is tempting to believe that safe Rust is protected by a theorem of the same kind, only bigger. It is not, and the reason is in every Rust program you have written. `Vec::push`, `split_at_mut`, `Cell::set` and `Mutex::lock` are implemented with `unsafe` code: raw pointers and operations that the type checker does not check. A proof by induction on typing derivations says nothing about code that has no typing derivation. Yet we say, rightly, that safe Rust has no undefined behavior.

This chapter explains what that sentence means and how it was proved. The key move is to stop asking "does this code have a derivation?" and to ask instead "does this code *behave* like a value of its type?". That is **semantic typing**, and its main tool is the **logical relation**. We build one in Lean for a small language with an unchecked primitive, the analogue of an `unsafe` operation, and verify a library that uses it safely. Then we read `unsafe` Rust as a proof obligation, and meet the logic in which Rust's soundness was actually proved: separation logic, and in particular Iris. At the end, affinity reappears in an unexpected place.

In this chapter you will learn:

- why syntactic soundness cannot account for safe abstractions built with `unsafe`, such as `split_at_mut`, `Vec` and `Cell`;
- what semantic typing is: types as predicates on values, a logical relation and its fundamental theorem, proved in Lean for a language with an unchecked primitive;
- how to read `unsafe` as a proof obligation, with `SAFETY:` comments, safety invariants and validity invariants;
- the basics of separation logic (heaps, points-to, the separating conjunction, the frame rule), with its laws proved in Lean;
- what RustBelt proved, with λRust, Iris and the lifetime logic, and what Stacked Borrows, Tree Borrows and Miri are for;
- what Rust's types do not guarantee, and why the logic used to prove Rust sound is *affine*.

32.1 Syntactic soundness and its limits

What Chapter 9 proved

Chapter 9 had a typing judgment $\Gamma \vdash e : \tau$, defined by rules, and a small-step semantics $e \to e'$. Type safety came in two lemmas: **progress** (a closed well-typed term is a value or can step) and **preservation** (stepping preserves the type), both proved by induction on the typing derivation, one case per rule. This is called **syntactic** type soundness, because "well typed" means "has a derivation built from a fixed list of rules". Andrew Wright and Matthias Felleisen made it the standard method in the 1990s, and Chapter 28 stated the same two lemmas for our borrow calculus.

The closed world of the rules

Syntactic soundness is a theorem about the terms that have derivations, and about nothing else. A term without a derivation gets no guarantee, even if it is harmless. With rule T-If of Chapter 1, the term

$$\texttt{if true then } 1 \texttt{ else false}$$

has no type, since its branches have different types. Yet it runs to 1, and anyone who uses it as a number is safe. The type system is *conservative*: it reasons about the shape of the text, and rejects some safe programs.

For a toy language this does not matter. For Rust it matters a great deal, because the safe programs that the rules reject include the standard library. Splitting a mutable slice into two mutable halves is obviously safe, as long as the halves do not overlap:

```
pub fn naive_split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    let len = slice.len();
    (&mut slice[..mid], &mut slice[mid..len])
}
```

```
error[E0499]: cannot borrow `*slice` as mutable more than once at a time
 --> compile_fail/ch32_naive_split.rs:8:30
  |
6 | pub fn naive_split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
  |                                  - let's call the lifetime of this reference `'1`
7 |     let len = slice.len();
8 |     (&mut slice[..mid], &mut slice[mid..len])
  |     ------------------------^^^^^-----------
  |     |     |                 |
  |     |     |                 second mutable borrow occurs here
  |     |     first mutable borrow occurs here
  |     returning this value requires that `*slice` is borrowed for `'1`
  |
  = help: use `.split_at_mut(position)` to obtain two mutable non-overlapping sub-slices
```

The borrow checker sees two mutable borrows of `*slice`, both returned, so both must last for the lifetime `'1`. That breaks aliasing XOR mutation (Chapter 26); in the calculus of Chapter 28, a mutable reference is not `Copy`, so the context cannot be split to give it to both reborrows. The checker has no way to know that `slice[..mid]` and `slice[mid..len]` are *disjoint*. Its hint suggests the standard library's `split_at_mut`, which is implemented with `unsafe`.

Two ways out

To prove that programs using `split_at_mut` are safe, we could make it a *primitive*, with its own typing and reduction rules, and redo progress and preservation. That works for one function, but the standard library has hundreds of them (`Vec`, `Box`, `Rc`, `Cell`, `RefCell`, `Mutex`, threads, channels) and crates.io has many more. Every library would need a new language and a new proof, and the proofs would not compose: in the "leakpocalypse" of Chapter 24, `Rc` and the old `thread::scoped` each looked sound on its own, and were unsound together.

The other way is to change what "well typed" means: not "has a derivation" but "behaves like a value of its type, in every context". That is a property of behavior, so it makes sense for any code, including code with raw pointers. The typing rules become *lemmas* about it, and a library written with `unsafe` can be proved to have it by hand, once.

32.2 Semantic types: types as predicates

Truth and provability, once more

You have seen this move before. Chapter 4 had provability, defined by rules, and truth, defined by truth tables, and proved that everything provable is true. The pattern:

- a **syntactic** relation, written $\vdash$, defined by rules and checked by a program;
- a **semantic** relation, written $\vDash$, defined by what things *mean*;
- a **soundness theorem**: $\vdash$ implies $\vDash$.

For types, $\Gamma \vDash e : \tau$ says that whenever e runs with good values for its variables, it produces a good value of type τ. The soundness theorem is called the **fundamental theorem** of the logical relation, and type safety follows from it. $\vdash$ is decidable, and a compiler checks it; $\vDash$ is not, but it is *open*: any code can be proved to satisfy it, by any argument.

A language with an unchecked primitive

We need a language with something that plays the role of `unsafe`. The file `code/lean/Book/Ch32/Language.lean` defines one: naturals, booleans, addition, comparison, `if`, functions, and one more operation.

```
/-- Types: natural numbers, booleans and functions. -/
inductive Ty where
  | nat
  | bool
  | arr (dom cod : Ty)
  deriving DecidableEq, Repr

/-- Terms. Variables are de Bruijn indices: `var 0` is the nearest enclosing
`lam`, `var 1` the one around it, and so on. -/
inductive Expr where
  | var (i : ℕ)
  | nat (n : ℕ)
  | bool (b : Bool)
  | add (e₁ e₂ : Expr)
  | le (e₁ e₂ : Expr)       -- `e₁ ≤ e₂`, a boolean
  | ite (c t f : Expr)      -- `if c then t else f`
  | lam (body : Expr)       -- a function whose parameter is `var 0` in `body`
  | app (f a : Expr)
```

```
  | subRaw (e₁ e₂ : Expr)   -- unchecked `e₁ - e₂`: stuck when `e₁ < e₂`
  deriving Repr
```

Variables are de Bruijn indices, as in Chapters 7 and 9. The odd one out is `subRaw e₁ e₂`, **unchecked subtraction**, which simply gets stuck when $e_1 < e_2$. It models an operation with a precondition, like Rust's `u32::unchecked_sub`, whose documentation says that overflow is undefined behavior. A formal semantics models undefined behavior the standard way: no rule applies, and the program is stuck.

The semantics is **big-step** with **environments**, rather than the small-step substitution semantics of Chapter 9: $\rho \vdash e \Downarrow v$ says that e, run with the variable values ρ, produces v. A function value is a **closure**, a body with the environment it was created in, as in the `stlc` crate of Chapter 8.

```
/-- Values: numbers, booleans and closures. A closure is a function body
together with the environment it was created in. -/
inductive Val where
  | nat (n : ℕ)
  | bool (b : Bool)
  | clo (env : List Val) (body : Expr)
  deriving Repr
```

```
/-- Big-step semantics. `Eval ρ e v` says: in the environment `ρ` (the values
of the variables, nearest first), the term `e` evaluates to the value `v`. -/
inductive Eval : List Val → Expr → Val → Prop where
  | var {ρ i v} : ρ[i]? = some v → Eval ρ (.var i) v
  | nat {ρ n} : Eval ρ (.nat n) (.nat n)
  | bool {ρ b} : Eval ρ (.bool b) (.bool b)
  | add {ρ e₁ e₂ n₁ n₂} :
      Eval ρ e₁ (.nat n₁) → Eval ρ e₂ (.nat n₂) →
      Eval ρ (.add e₁ e₂) (.nat (n₁ + n₂))
  | le {ρ e₁ e₂ n₁ n₂} :
      Eval ρ e₁ (.nat n₁) → Eval ρ e₂ (.nat n₂) →
      Eval ρ (.le e₁ e₂) (.bool (decide (n₁ ≤ n₂)))
  | iteTrue {ρ c t f v} :
      Eval ρ c (.bool true) → Eval ρ t v → Eval ρ (.ite c t f) v
  | iteFalse {ρ c t f v} :
      Eval ρ c (.bool false) → Eval ρ f v → Eval ρ (.ite c t f) v
  | lam {ρ body} : Eval ρ (.lam body) (.clo ρ body)
  | app {ρ f a ρ' body w v} :
      Eval ρ f (.clo ρ' body) → Eval ρ a w → Eval (w :: ρ') body v →
      Eval ρ (.app f a) v
  | subRaw {ρ e₁ e₂ n₁ n₂} :
      Eval ρ e₁ (.nat n₁) → Eval ρ e₂ (.nat n₂) → n₂ ≤ n₁ →
      Eval ρ (.subRaw e₁ e₂) (.nat (n₁ - n₂))
```

Rule `app` evaluates the function to a closure `clo ρ' body`, the argument to `w`, and runs the body in `w :: ρ'`. The interesting rule is the last one:

$$\frac{\rho \vdash e_1 \Downarrow n_1 \qquad \rho \vdash e_2 \Downarrow n_2 \qquad n_2 \le n_1}{\rho \vdash \texttt{subRaw}\; e_1\; e_2 \Downarrow n_1 - n_2}\ \text{E-SubRaw}$$

The third premise is the precondition. When it fails, no rule applies and no derivation exists: that is "stuck" in a big-step semantics. The typing rules are the familiar ones:

```
/-- The syntactic typing rules. There is deliberately no rule for `subRaw`:
the type system cannot check its precondition `e₂ ≤ e₁`. -/
inductive HasType : List Ty → Expr → Ty → Prop where
  | var {Γ i τ} : Γ[i]? = some τ → HasType Γ (.var i) τ
  | nat {Γ n} : HasType Γ (.nat n) .nat
  | bool {Γ b} : HasType Γ (.bool b) .bool
  | add {Γ e₁ e₂} :
```

```
      HasType Γ e₁ .nat → HasType Γ e₂ .nat → HasType Γ (.add e₁ e₂) .nat
  | le {Γ e₁ e₂} :
      HasType Γ e₁ .nat → HasType Γ e₂ .nat → HasType Γ (.le e₁ e₂) .bool
  | ite {Γ c t f τ} :
      HasType Γ c .bool → HasType Γ t τ → HasType Γ f τ →
      HasType Γ (.ite c t f) τ
  | lam {Γ body σ τ} :
      HasType (σ :: Γ) body τ → HasType Γ (.lam body) (.arr σ τ)
  | app {Γ f a σ τ} :
      HasType Γ f (.arr σ τ) → HasType Γ a σ → HasType Γ (.app f a) τ
```

There is **no rule for `subRaw`**: the type system cannot know the values of e_1 and e_2 before the program runs, so it rejects every use of the primitive. That is precisely the status of `unsafe` operations in Rust. (The λ carries no annotation; since `HasType` is a relation, rule `lam` may choose any σ.) Here are two "library functions" built with the primitive:

```
/-- Saturating subtraction, `fun x y => if y ≤ x then x - y else 0`, written
with the unchecked primitive. Inside the body, `var 1` is `x` and `var 0`
is `y`. The `if` establishes the precondition of `subRaw` before calling it. -/
def satSub : Expr :=
  .lam (.lam (.ite (.le (.var 0) (.var 1)) (.subRaw (.var 1) (.var 0)) (.nat 0)))

/-- The careless version: it calls the primitive without checking. -/
def badSub : Expr :=
  .lam (.lam (.subRaw (.var 1) (.var 0)))

/-- Apply a two-argument function to two numbers. -/
def call₂ (f : Expr) (x y : ℕ) : Expr := .app (.app f (.nat x)) (.nat y)
```

`satSub` is saturating subtraction, and calls the primitive only after checking its precondition; `badSub` does not check. Neither has a type. The file also defines an interpreter `run`, with fuel like the normalizer of Chapter 7, which returns a value, `stuck` or `outOfFuel`, and proves it agrees with `Eval` (`run_sound`):

```
/-- info: Ch32.Outcome.value (Ch32.Val.nat 2) -/
#guard_msgs in
#eval run 10 [] (call₂ satSub 5 3)

/-- info: Ch32.Outcome.value (Ch32.Val.nat 0) -/
#guard_msgs in
#eval run 10 [] (call₂ satSub 3 5)

/-- info: Ch32.Outcome.value (Ch32.Val.nat 2) -/
#guard_msgs in
#eval run 10 [] (call₂ badSub 5 3)

/-- info: Ch32.Outcome.stuck -/
#guard_msgs in
#eval run 10 [] (call₂ badSub 3 5)
```

`badSub` gets stuck on `3` and `5`. We want a notion of typing that accepts `satSub`, rejects `badSub`, and accepts everything `HasType` accepts.

Base types: a value of the right shape

A good value of type `nat` is any number; of type `bool`, a boolean. We write $\mathcal{V}[\![\tau]\!]$ for the good values of type τ (double brackets traditionally mean "the meaning of"), and $\mathcal{E}[\![\tau]\!]$ for the closed terms that run to one:

$$\mathcal{V}[\![\mathtt{nat}]\!] = \{\, \mathit{nat}\ n \mid n \in \mathbb{N} \,\} \qquad \mathcal{E}[\![\tau]\!] = \{\, e \mid \exists v.\, e \Downarrow v \ \text{ and } \ v \in \mathcal{V}[\![\tau]\!] \,\}$$

These two definitions already separate the two notions of typing on the term from the beginning of the chapter:

```
/-- `if true then 1 else false`. -/
def mixed : Expr := .ite (.bool true) (.nat 1) (.bool false)

/-- No derivation exists: rule `ite` would need both branches at type `nat`. -/
theorem mixed_not_typed : ¬ HasType [] mixed .nat := by
  intro h
  cases h with
  | ite _ _ hf => cases hf

/-- But the term does run to a number. -/
theorem mixed_sem : SemTyped [] mixed .nat := by
  intro ρ _
  exact ⟨.nat 1, .iteTrue .bool .nat, 1, rfl⟩
```

The first proof shows that *no* derivation exists. `cases h` asks which rule could have produced `h`; only rule `ite` fits, and it yields a derivation `hf` of `HasType [] (.bool false) .nat`. A second `cases hf` finds no rule that concludes such a judgment, so there is nothing left to prove. This is proof by inversion, as in Chapter 28. The second proof exhibits the evaluation `.iteTrue .bool .nat` to `nat 1`, and `⟨1, rfl⟩` shows that `nat 1` is a number. (`SemTyped`, defined below, says for a closed term exactly that it is in $\mathcal{E}[\![\tau]\!]$.)

Function types: the logical step

What is a good value of type `nat` → `nat`? "A closure" is not enough: `badSub` applied to `3` is a closure, and gets stuck when applied to `5`. A good function turns good arguments into good results:

$$\mathcal{V}[\![\sigma \to \tau]\!] = \{\, \mathit{clo}(\rho, e) \mid \forall w \in \mathcal{V}[\![\sigma]\!].\ \exists r.\ (w :: \rho) \vdash e \Downarrow r \ \text{ and } \ r \in \mathcal{V}[\![\tau]\!] \,\}$$

The meaning of the function type is defined from the meanings of its parts, and has the shape of an implication: "if the input is good, the output is good". That is why such a family of predicates is called a **logical relation** (a *logical predicate*, strictly, since it is unary; binary ones relate two programs and are used to prove equivalences). It is the BHK reading of $A \to B$ from Chapter 5: a method that turns proofs of A into proofs of B. The technique goes back to proofs of normalization for typed λ-calculi in the 1960s and 1970s, starting with work by William Tait. In Lean, the definition is a recursion on the type:

```
/-- `V τ v`: the value `v` is a good value of type `τ`. -/
def V : Ty → Val → Prop
  | .nat, v => ∃ n, v = .nat n
  | .bool, v => ∃ b, v = .bool b
  | .arr σ τ, v =>
    ∃ ρ body, v = .clo ρ body ∧
      ∀ w, V σ w → ∃ r, Eval (w :: ρ) body r ∧ V τ r
```

`V σ w` appears to the *left* of an arrow in the case for `.arr σ τ`. As an inductive predicate, this would be a negative occurrence, which Lean rejects (Chapter 11, positivity). As a recursion on the type it is fine: `V (.arr σ τ)` refers to `V σ` and `V τ`, about smaller types. This is why logical relations are defined by recursion on types (or, with step-indexing, on types and steps).

Open terms

A term with free variables runs in an environment, and semantic typing quantifies over every environment that gives each variable a good value of its type (the set $\mathcal{G}[\![\Gamma]\!]$):

$$\Gamma \vDash e : \tau \quad \text{iff} \quad \forall \rho \in \mathcal{G}[\![\Gamma]\!].\ \rho \vdash e \in \mathcal{E}[\![\tau]\!]$$

```
/-- `E τ ρ e`: in the environment `ρ`, the term `e` runs to a good value of
type `τ`. -/
def E (τ : Ty) (ρ : List Val) (e : Expr) : Prop :=
  ∃ v, Eval ρ e v ∧ V τ v
```

```
/-- `G Γ ρ`: the environment `ρ` gives every variable of `Γ` a good value of
its type. -/
def G : List Ty → List Val → Prop
  | [], [] => True
  | τ :: Γ, v :: ρ => V τ v ∧ G Γ ρ
  | _, _ => False

/-- Semantic typing, written `Γ ⊨ e : τ` in the text: in every environment that
fits `Γ`, the term `e` runs to a good value of type `τ`. -/
def SemTyped (Γ : List Ty) (e : Expr) (τ : Ty) : Prop :=
  ∀ ρ, G Γ ρ → E τ ρ e
```

A small lemma in the file, `G_lookup`, says that if `G Γ ρ` holds and `Γ` gives variable `i` the type `τ`, then `ρ` has at position `i` a value in `V τ`.

Compatibility lemmas

The fundamental theorem is proved by induction on derivations, and each case is a lemma of its own: a typing rule with $\vdash$ replaced by $\vDash$. These are the **compatibility lemmas**:

```
theorem compat_var {Γ i τ} (h : Γ[i]? = some τ) : SemTyped Γ (.var i) τ := by
  intro ρ hG
  obtain ⟨v, hv, hV⟩ := G_lookup hG h
  exact ⟨v, .var hv, hV⟩
```

```
theorem compat_nat {Γ n} : SemTyped Γ (.nat n) .nat :=
  fun _ _ => ⟨.nat n, .nat, n, rfl⟩

theorem compat_bool {Γ b} : SemTyped Γ (.bool b) .bool :=
  fun _ _ => ⟨.bool b, .bool, b, rfl⟩
```

For a variable, `G_lookup` gives the value and its goodness, and `Eval.var` turns the lookup into an evaluation. A constant evaluates to itself. Addition:

```
theorem compat_add {Γ e₁ e₂} (h₁ : SemTyped Γ e₁ .nat)
    (h₂ : SemTyped Γ e₂ .nat) : SemTyped Γ (.add e₁ e₂) .nat := by
  intro ρ hG
  obtain ⟨_, ev₁, n₁, rfl⟩ := h₁ ρ hG
  obtain ⟨_, ev₂, n₂, rfl⟩ := h₂ ρ hG
  exact ⟨.nat (n₁ + n₂), .add ev₁ ev₂, n₁ + n₂, rfl⟩
```

Applied to `ρ` and `hG`, each hypothesis gives a value, its evaluation, and the fact that it is good at `nat`, which unfolds to "it is `nat n` for some `n`". The `obtain` pattern `⟨_, ev₁, n₁, rfl⟩` takes all of this apart, the `rfl` substituting `nat n₁` for the value. `Eval.add` then evaluates the sum, which is a number. For `ite`, the boolean computed by the condition decides which branch's hypothesis to use:

```
theorem compat_ite {Γ c t f τ} (hc : SemTyped Γ c .bool)
    (ht : SemTyped Γ t τ) (hf : SemTyped Γ f τ) :
    SemTyped Γ (.ite c t f) τ := by
  intro ρ hG
  obtain ⟨_, evc, b, rfl⟩ := hc ρ hG
  cases b with
  | true =>
    obtain ⟨v, ev, hv⟩ := ht ρ hG
    exact ⟨v, .iteTrue evc ev, hv⟩
  | false =>
    obtain ⟨v, ev, hv⟩ := hf ρ hG
    exact ⟨v, .iteFalse evc ev, hv⟩
```

The lemmas for functions are where the definition of `V` at function type does its work:

```
theorem compat_lam {Γ body σ τ} (h : SemTyped (σ :: Γ) body τ) :
    SemTyped Γ (.lam body) (.arr σ τ) := by
  intro ρ hG
  refine ⟨.clo ρ body, .lam, ρ, body, rfl, ?_⟩
  intro w hw
  exact h (w :: ρ) ⟨hw, hG⟩
```

`lam body` evaluates to `clo ρ body`, and we must show that for every good argument `w` the body runs to a good result in `w :: ρ`. Hypothesis `h` gives exactly that, because `w :: ρ` fits `σ :: Γ`: the anonymous constructor `⟨hw, hG⟩` proves `G (σ :: Γ) (w :: ρ)`, which unfolds to `V σ w ∧ G Γ ρ`.

```
theorem compat_app {Γ f a σ τ} (hf : SemTyped Γ f (.arr σ τ))
    (ha : SemTyped Γ a σ) : SemTyped Γ (.app f a) τ := by
  intro ρ hG
  obtain ⟨_, evf, ρ', body, rfl, hbody⟩ := hf ρ hG
  obtain ⟨w, eva, hw⟩ := ha ρ hG
  obtain ⟨r, evr, hr⟩ := hbody w hw
  exact ⟨r, .app evf eva evr, hr⟩
```

The function evaluates to a good closure, which carries the promise `hbody` ("give me a good argument and I run to a good result"); the argument evaluates to a good `w`; we call the promise, and `Eval.app` assembles the three evaluations. Compare Chapter 9, where preservation needed a substitution lemma: here the definition of `V` at function type *is* the statement we need.

The fundamental theorem

```
/-- Every syntactically well-typed term is semantically well typed. -/
theorem fundamental {Γ e τ} (h : HasType Γ e τ) : SemTyped Γ e τ := by
  induction h with
  | var h => exact compat_var h
  | nat => exact compat_nat
  | bool => exact compat_bool
  | add _ _ ih₁ ih₂ => exact compat_add ih₁ ih₂
  | le _ _ ih₁ ih₂ => exact compat_le ih₁ ih₂
  | ite _ _ _ ihc iht ihf => exact compat_ite ihc iht ihf
  | lam _ ih => exact compat_lam ih
  | app _ _ ihf iha => exact compat_app ihf iha
```

`induction h` produces one case per typing rule, with the induction hypotheses (`ih₁`, `ihf`, ...) as semantic typings of the premises, and each case is closed by its compatibility lemma. Type safety is a corollary, since the empty environment fits the empty context (`G [] []` is `True`):

```
/-- Type safety as a corollary: a closed well-typed term runs to a value of
its type. It never gets stuck (and, in this language, it always finishes). -/
```

```
theorem safety {e τ} (h : HasType [] e τ) : ∃ v, Eval [] e v ∧ V τ v :=
  fundamental h [] trivial
```

This says a little more than Chapter 9: evaluation *terminates*, since a big-step derivation is a finite tree. That holds because this language, like the simply typed λ-calculus, has no recursion. For a language with loops (Rust), one uses a small-step semantics and a *step-indexed* logical relation, which promises only "never stuck, however many steps you run". Iris, below, has step-indexing built in.

32.3 Verifying a library that uses the primitive

The semantic system accepts code that uses the unchecked primitive correctly. The syntactic one does not:

```
/-- `satSub` has no syntactic type at all: its body uses `subRaw`. -/
theorem satSub_not_typed (τ : Ty) : ¬ HasType [] satSub τ := by
  intro h
  cases h with
  | lam h =>
    cases h with
    | lam h =>
      cases h with
      | ite _ ht _ => cases ht
```

Three inversions (two `lam`s and an `ite`) leave a derivation `ht` for `subRaw ...`, for which no rule exists, so the last `cases ht` has no cases. For the semantic proof we need one fact: in an environment that starts with numbers `y` and `x`, the test `var 0 ≤ var 1` evaluates to the boolean `decide (y ≤ x)` (`decide` turns a decidable proposition into a `Bool`, as in Chapter 5). The two lookups hold by `rfl`, since `(nat y :: nat x :: ρ)[0]?` computes to `some (nat y)`.

```
/-- The comparison `var 0 ≤ var 1` in an environment that starts with two
numbers `y` and `x`. -/
theorem eval_le_vars {y x : ℕ} {ρ : List Val} :
    Eval (.nat y :: .nat x :: ρ) (.le (.var 0) (.var 1)) (.bool (decide (y ≤ x))) :=
  .le (.var rfl) (.var rfl)
```

```
/-- The proof obligation of the library, discharged once and for all:
`satSub` is a good function from two numbers to a number. -/
theorem satSub_sem : SemTyped [] satSub (.arr .nat (.arr .nat .nat)) := by
  intro ρ _
  -- `satSub` evaluates to a closure; we must show it maps numbers to good
  -- functions from numbers to numbers.
  refine ⟨_, .lam, ρ, _, rfl, ?_⟩
  rintro _ ⟨x, rfl⟩
  refine ⟨_, .lam, _, _, rfl, ?_⟩
  rintro _ ⟨y, rfl⟩
  -- The body `if y ≤ x then subRaw x y else 0` runs in `[y, x] ++ ρ`.
  by_cases hyx : y ≤ x
  · -- The `then` branch: `hyx` is exactly the precondition of `subRaw`.
    have hc := eval_le_vars (ρ := ρ) (y := y) (x := x)
    rw [decide_eq_true hyx] at hc
    exact ⟨.nat (x - y), .iteTrue hc (.subRaw (.var rfl) (.var rfl) hyx), _, rfl⟩
  · -- The `else` branch never calls the primitive.
    have hc := eval_le_vars (ρ := ρ) (y := y) (x := x)
    rw [decide_eq_false hyx] at hc
    exact ⟨.nat 0, .iteFalse hc .nat, 0, rfl⟩
```

Step by step:

- `intro ρ _` takes any environment that fits the empty context.
- The first `refine` says that `satSub` evaluates (rule `Eval.lam`) to a closure, and leaves, as `?_`, the condition of `V (.arr .nat (.arr .nat .nat))`.
- `rintro _ ⟨x, rfl⟩` introduces the first argument with the proof that it is a number; the `rfl` pattern replaces it by `nat x`. The inner `lam` is handled the same way, with second argument `nat y`. What remains: the body `if y ≤ x then subRaw x y else 0` runs, in `nat y :: nat x :: ρ`, to a number.
- `by_cases hyx : y ≤ x` splits on the condition. In the first case, `have` takes the evaluation `hc` of the test from `eval_le_vars`, `rw` rewrites `decide (y ≤ x)` in it to `true` (lemma `decide_eq_true`), and we assemble `.iteTrue hc (.subRaw (.var rfl) (.var rfl) hyx)`. The last argument of `.subRaw` is the proof of the precondition $n_2 \le n_1$ demanded by rule E-SubRaw, and we have it because we are in the branch where the program checked it. In the second case the test is `false` (`decide_eq_false`), the `else` branch is the constant `0`, and the primitive is never called.

This proof is the formal version of a `SAFETY:` comment. The sentence you would write in Rust next to the call, "`y <= x` because the `if` just checked it", is the argument `hyx`. The careless library fails the same test, and Lean can prove it:

```
/-- The careless library is not semantically well typed: called with `3` and
`5`, its body is stuck. -/
theorem badSub_not_sem : ¬ SemTyped [] badSub (.arr .nat (.arr .nat .nat)) := by
  intro h
  -- Run `badSub` in the empty environment: it must give a good closure.
  obtain ⟨_, ev, _, _, rfl, hf⟩ := h [] trivial
  cases ev
  -- Apply it to `3`: the result must be a good closure again.
  obtain ⟨_, ev₃, _, _, rfl, hg⟩ := hf (.nat 3) ⟨3, rfl⟩
  cases ev₃
  -- Apply that to `5`: the body `subRaw x y` must run to a value.
  obtain ⟨_, ev₅, _⟩ := hg (.nat 5) ⟨5, rfl⟩
  -- The only rule for `subRaw` needs `5 ≤ 3`.
  cases ev₅ with
  | subRaw ev₁ ev₂ hle =>
    cases ev₁ with
    | var h₁ =>
      cases ev₂ with
      | var h₂ =>
        simp only [List.getElem?_cons_succ, List.getElem?_cons_zero,
          Option.some.injEq, Val.nat.injEq] at h₁ h₂
        omega
```

Assuming `badSub` is semantically well typed, we run it: `cases ev` finds that the evaluation used rule `Eval.lam`, which fixes the closure. Applying it to `nat 3` gives another good closure, and applying that to `nat 5` must give an evaluation `ev₅` of `subRaw (var 1) (var 0)` in `[nat 5, nat 3]`. Inverting `ev₅` leaves only rule `subRaw`, with side condition `hle : n₂ ≤ n₁`; inverting the two lookups shows $n_1 = 3$ and $n_2 = 5$ (`simp only` computes the lookups), and `omega` refutes $5 \le 3$.

Linking a verified library with type-checked clients

A client of the library should not need its own proof. It should be enough to type-check it against the library's *signature*, as rustc checks your calls to `split_at_mut` without looking inside. We model a client as a term with one free variable of the library's type, and linking as binding that variable:

```
/-- If a library is semantically well typed at `σ`, and a client type-checks
with one variable of type `σ` (the library's signature, nothing more), then
the linked program `(fun lib => client) library` runs to a good value. -/
theorem link {lib c : Expr} {σ τ : Ty}
    (hlib : SemTyped [] lib σ) (hc : HasType [σ] c τ) :
    ∃ v, Eval [] (.app (.lam c) lib) v ∧ V τ v := by
  obtain ⟨w, evw, hw⟩ := hlib [] trivial
  obtain ⟨v, evv, hv⟩ := fundamental hc [w] ⟨hw, trivial⟩
  exact ⟨v, .app .lam evw evv, hv⟩
```

The library's semantic typing gives a good value `w`; the client's *syntactic* typing, through the fundamental theorem, runs the client in `[w]`. A concrete client:

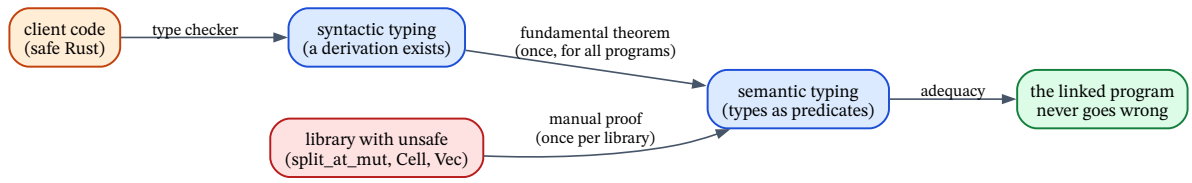
```
/-- A client that only knows `sub : nat → nat → nat` (it is `var 0`):
`sub (sub 10 3) 4`. -/
def client : Expr :=
  .app (.app (.var 0) (.app (.app (.var 0) (.nat 10)) (.nat 3))) (.nat 4)

theorem client_typed : HasType [.arr .nat (.arr .nat .nat)] client .nat :=
  .app (.app (.var rfl) (.app (.app (.var rfl) .nat) .nat)) .nat

/-- The linked program is safe, with no further proof about `satSub`. -/
theorem client_safe : ∃ v, Eval [] (.app (.lam client) satSub) v ∧ V .nat v :=
  link satSub_sem client_typed

/-- info: Ch32.Outcome.value (Ch32.Val.nat 3) -/
#guard_msgs in
#eval run 20 [] (.app (.lam client) satSub)
```

The client's typing is a proof term built from the constructors of `HasType`. `client_safe` costs one line: the work about `satSub` was done once, for every client. The diagram shows this structure, which is also that of the proof of Rust's soundness.



The structure of a semantic soundness proof. Clients are type-checked syntactically; the fundamental theorem, proved once, turns their derivations into semantic typings; libraries that use unsafe operations are proved semantically well typed by hand, once each; and adequacy turns semantic typing into safety of the linked program.

32.4 `unsafe` as a proof obligation

Now the Rust side. The code is in `code/rust/examples/ch32_unsafe_abstractions.rs`.

What `unsafe` unlocks

`unsafe` does not turn off the type checker or the borrow checker. It unlocks five operations, each with preconditions the compiler cannot verify: dereferencing a raw pointer, calling an `unsafe` function, accessing a mutable `static`, implementing an `unsafe` trait, and reading a `union` field. Outside `unsafe`, the safe fragment has no rule for them, as `HasType` has none for `subRaw`:

```
pub fn sub_unchecked(x: u32, y: u32) -> u32 {
    x.unchecked_sub(y)
}
```

```
error[E0133]: call to unsafe function `core::num::<impl u32>::unchecked_sub` is unsafe and
requires unsafe function or block
 --> compile_fail/ch32_unsafe_call.rs:8:5
  |
8 |     x.unchecked_sub(y)
  |     ^^^^^^^^^^^^^^^^^^ call to unsafe function
  |
  = note: consult the function's documentation for information on how to avoid undefined
behavior
```

`unchecked_sub` is the real counterpart of our primitive, and the note says where its precondition lives: in the documentation, in English. The keyword has three roles:

- an **`unsafe` block** says: *I* discharge the obligations of the operations inside (a proof like `satSub_sem` happens here);
- an **`unsafe fn`** says: *the caller* must establish a precondition, written in a `# Safety` section of its documentation;
- an **`unsafe impl`** of an `unsafe trait` promises a property that other unsafe code relies on, as `unsafe impl Sync for T` promises that sharing `&T` across threads is safe.

The convention that makes this workable is the **`SAFETY:` comment** before every `unsafe` block, stating the preconditions involved and why they hold. It is a proof, in English, of the premises of a rule the compiler does not know. Here are the Rust twins of our Lean libraries:

```
/// The Rust twin of `satSub` in `Book/Ch32/Language.lean`.
fn sat_sub(x: u32, y: u32) -> u32 {
    if y <= x {
        // SAFETY: `unchecked_sub` requires that `x - y` does not overflow,
        // that is, `y <= x`, which the `if` has just established.
        unsafe { x.unchecked_sub(y) }
    } else {
        0
    }
}

/// The twin of `badSub`. It compiles and it passes every test below, but it
/// is UNSOUND: it is a safe function, so every caller may pass any `x` and
/// `y`, and `bad_sub(3, 5)` is undefined behavior. No `SAFETY:` comment can
/// be written for this block, because nothing establishes `y <= x`.
fn bad_sub(x: u32, y: u32) -> u32 {
    unsafe { x.unchecked_sub(y) }
}
```

Both compile, and the example's tests pass for both, because they call `bad_sub` only on inputs where it is safe:

```
sat_sub(5, 3) = 2, sat_sub(3, 5) = 0
bad_sub(5, 3) = 2 (never call bad_sub(3, 5))
```

A function without `unsafe` in its signature promises every caller that it is safe *for all inputs*, the same "for all" as in the definition of $\mathcal{V}[\![\sigma \to \tau]\!]$. No finite set of tests establishes it, and `bad_sub` is unsound: it is not semantically well typed at `fn(u32, u32) -> u32`. Tools only see symptoms: Clippy's `undocumented_unsafe_blocks` lint flags exactly one block in this example, the one in `bad_sub`, which

has no `SAFETY:` comment; and in a debug build, `bad_sub(3, 5)` aborts with "unsafe precondition(s) violated: u32::unchecked_sub cannot overflow", a check the standard library calls optional. In a release build it is gone.

An `unsafe fn` passes the obligation up:

```
/// Sum the first `n` readings, without bounds checks.
///
/// # Safety
///
/// The caller must guarantee `n <= readings.len()`.
unsafe fn sum_prefix_unchecked(readings: &[u64], n: usize) -> u64 {
    let mut total = 0;
    for i in 0..n {
        // SAFETY: `i < n <= readings.len()`, by the loop bound and by the
        // precondition of this function, which our caller promised.
        total += unsafe { *readings.get_unchecked(i) };
    }
    total
}

fn sum_first_three(readings: &[u64]) -> Option<u64> {
    if readings.len() < 3 {
        return None;
    }
    // SAFETY: `3 <= readings.len()`, checked just above.
    Some(unsafe { sum_prefix_unchecked(readings, 3) })
}
```

```
sum_first_three([5, 6, 7, 8]) = Some(18)
sum_first_three([5, 6]) = None
```

Since the 2024 edition, an unsafe operation in the body of an `unsafe fn` still needs its own `unsafe` block: otherwise the lint `unsafe_op_in_unsafe_fn` warns that "an unsafe function restricts its caller, but its body is safe by default". So each operation inside gets its own block and justification. The precondition `n <= readings.len()` is what Lean would make an *argument*: recall `first` in Chapter 1, which took a proof `h : 0 < xs.size`. In Lean the caller supplies a proof that the kernel checks; in Rust, an `unsafe` block and a comment that a reviewer checks.

Safety invariants and validity invariants

The **validity invariant** of a type must hold at *all* times, even inside unsafe code, because the compiler relies on it: a `bool` is 0 or 1, a reference is non-null and aligned, a `NonZeroU32` is not zero. Violating it, even briefly, is undefined behavior at once. The compiler uses these facts to lay out data, as in Chapter 11's niche optimization.

The **safety invariant** must hold whenever *safe* code can observe the value, and may be broken temporarily inside a library: a `String` holds UTF-8; a `Vec<T>` has `len <= capacity` with the first `len` elements initialized; the halves returned by `split_at_mut` do not overlap. Breaking it becomes undefined behavior when a safe method that trusts it does something only correct under it. In this chapter's terms, the safety invariant is the semantic type $\mathcal{V}[\![\tau]\!]$; the validity invariant is the much weaker property the compiler needs.

```
/// Safety invariant: a `String` must hold UTF-8. Breaking it is not
/// immediately undefined behavior, but safe methods may misbehave later.
```

```rust
fn shout(s: &str) -> String {
    let bytes: Vec<u8> = s.bytes().map(|b| b.to_ascii_uppercase()).collect();
    // SAFETY: `from_utf8_unchecked` requires valid UTF-8. `s` was valid
    // UTF-8, and `to_ascii_uppercase` only changes bytes in `b'a'..=b'z'`,
    // which never occur inside a multi-byte character (those bytes are all
    // `>= 0x80`), so the result is valid UTF-8 too.
    unsafe { String::from_utf8_unchecked(bytes) }
}

/// Validity invariant: a `NonZeroU32` is never 0, and the compiler relies on
/// it for the layout of `Option<NonZeroU32>`: 0 encodes `None`.
fn port(n: u32) -> Option<NonZeroU32> {
    if n == 0 {
        None
    } else {
        // SAFETY: `new_unchecked` requires `n != 0`, which the `if` checked.
        Some(unsafe { NonZeroU32::new_unchecked(n) })
    }
}
```

```
shout("safe abstraction") = "SAFE ABSTRACTION"
port(8080) = Some(8080), port(0) = None
size_of: u32 = 4, Option<u32> = 8, Option<NonZeroU32> = 4
```

`Option<u32>` needs a separate tag (8 bytes), while `Option<NonZeroU32>` takes 4, because 0 encodes `None`. That is why `NonZeroU32::new_unchecked(0)` is undefined behavior at once, while a `String` holding invalid UTF-8 is "only" a broken promise.

`split_at_mut`, our way

Here is the function the borrow checker rejected, written as the standard library writes it:

```rust
/// Split one unique borrow of a slice into unique borrows of its two halves.
///
/// The signature promises two `&mut` slices that do not overlap. The borrow
/// checker cannot verify that (to it, both halves are borrowed from `slice`),
/// so the body works with a raw pointer, and the `unsafe` block carries the
/// argument that the compiler cannot check.
pub fn my_split_at_mut<'a, T>(
    slice: &'a mut [T],
    mid: usize,
) -> (&'a mut [T], &'a mut [T]) {
    let len = slice.len();
    // This check is what makes the unsafe block sound. Without it, a caller
    // could pass `mid > len` and receive a slice that runs past the end.
    assert!(mid <= len, "mid > len");
    let ptr: *mut T = slice.as_mut_ptr();
    // SAFETY: `from_raw_parts_mut(p, n)` requires that `p` is non-null,
    // aligned, and valid for reads and writes of `n` initialized values of `T`
    // inside one allocation, and that nothing else accesses that memory while
    // the result is alive. `ptr.add(mid)` requires that the result stays in
    // the same allocation (or one past its end).
    // 1. `ptr` comes from `slice`, so it is non-null, aligned and points to
    //    `len` initialized values.
    // 2. `mid <= len` (checked above), so `ptr.add(mid)` is in bounds, and
    //    both ranges `[0, mid)` and `[mid, len)` lie inside `slice`.
    // 3. The two ranges are disjoint, so the two results never alias each
    //    other. This is the fact that the borrow checker cannot see.
    // 4. Both results live for `'a`, the lifetime of the unique borrow
    //    `slice`, so no one else can use the memory while they are alive.
    unsafe {
        (
            std::slice::from_raw_parts_mut(ptr, mid),
            std::slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
```

```
    }
}
```

The `SAFETY:` comment lists the preconditions of `slice::from_raw_parts_mut` and `pointer::add`, then discharges them. Point 2 relies on the `assert!`: the function establishes its own precondition, so a bad `mid` causes a panic, which is safe, instead of undefined behavior. Point 3 is the fact the borrow checker could not see. Point 4 explains why one lifetime `'a` appears three times: while either half is alive, `slice` stays mutably borrowed. (The lifetime is written out for the discussion; elision gives the same signature.)

```
let mut samples = [3, 1, 4, 1, 5, 9, 2, 6];
let (left, right) = my_split_at_mut(&mut samples, 3);
left[0] = 30; // two unique borrows, used side by side
right[0] = 10;
left.swap(1, 2);
right.reverse();
println!("left  = {left:?}");
println!("right = {right:?}");
println!("all   = {samples:?}");
```

```
left  = [30, 4, 1]
right = [6, 2, 9, 5, 10]
all   = [30, 4, 1, 6, 2, 9, 5, 10]
```

In the terms of the diagram, `my_split_at_mut` is a library proved (informally) semantically well typed at its signature, and `main` is a client checked by rustc against that signature.

A `Cell` of our own

`Cell<T>` permits mutation through a shared reference, an apparent violation of aliasing XOR mutation that is sound because of an invariant its API protects:

```
/// A minimal `Cell`: mutation through a shared reference, on one thread.
///
/// The invariant that every method relies on: no reference into `value` ever
/// escapes a method, and no method runs code it does not control (such as a
/// destructor or a `clone`) while it is accessing `value`. So whenever a
/// method touches the contents, no other reference to them is alive.
pub struct MyCell<T> {
    value: UnsafeCell<T>,
}

impl<T> MyCell<T> {
    pub const fn new(value: T) -> Self {
        MyCell {
            value: UnsafeCell::new(value),
        }
    }

    /// Put `value` in the cell and return the old contents.
    pub fn replace(&self, value: T) -> T {
        // SAFETY: we create a `&mut T` to the contents for the duration of
        // `mem::replace`, which only moves bytes and runs no other code.
        // No other reference to the contents is alive: no method lets one
        // escape, and `MyCell` is not `Sync` (because `UnsafeCell` is not),
        // so no other thread can be inside a method of this cell right now.
        unsafe { std::mem::replace(&mut *self.value.get(), value) }
    }

    /// Store `value`. The old contents are dropped *after* the access ends:
    /// a destructor can run arbitrary code, including code that uses this
```

```
    /// very cell.
    pub fn set(&self, value: T) {
        let old = self.replace(value);
        drop(old);
    }

    pub fn into_inner(self) -> T {
        self.value.into_inner()
    }
}

impl<T: Copy> MyCell<T> {
    /// Copy the contents out. The bound `T: Copy` is part of the safety
    /// argument: copying is a plain bitwise read, while a `clone` could run
    /// code that calls `set` on this cell in the middle of our read.
    pub fn get(&self) -> T {
        // SAFETY: we read the contents through a raw pointer and copy them;
        // as in `replace`, no other reference to the contents is alive, and
        // no user code runs during the read.
        unsafe { *self.value.get() }
    }
}
```

`UnsafeCell<T>` is the one primitive through which Rust permits mutation behind a shared reference; `UnsafeCell::get` returns a raw pointer to the contents. The invariant in the doc comment (no reference escapes, no foreign code runs during an access) explains three details:

- `get` requires `T: Copy`. A copy runs no user code. If `get` called `clone` instead, the user's `clone` could call `set` on the same cell while we hold a reference into it. The trait bound is part of the safety argument.
- `set` drops the old value *after* the access, since a destructor is user code.
- `MyCell` is not `Sync`, without our saying so: `UnsafeCell` is not `Sync`, and auto traits propagate to the struct. So "no other thread is inside a method" is enforced by the type system:

```
pub fn bump_in_two_threads(counter: &MyCell<u32>) {
    std::thread::scope(|s| {
        s.spawn(|| counter.bump());
        counter.bump(); // if this compiled, it would be a data race
    });
}
```

```
error[E0277]: `UnsafeCell<u32>` cannot be shared between threads safely
  --> compile_fail/ch32_cell_not_sync.rs:22:17
   |
22 |         s.spawn(|| counter.bump());
   |           -----  ^^^^^^^^^^^^^^^^^ `UnsafeCell<u32>` cannot be shared between threads
safely
   |           |
   |           required by a bound introduced by this call
   |
   = help: within `MyCell<u32>`, the trait `Sync` is not implemented for `UnsafeCell<u32>`
note: required because it appears within the type `MyCell<u32>`
  --> compile_fail/ch32_cell_not_sync.rs:8:12
   |
 8 | pub struct MyCell<T> {
   |            ^^^^^^
   = note: required for `&MyCell<u32>` to implement `Send`
note: required because it's used within this closure
  --> compile_fail/ch32_cell_not_sync.rs:22:17
   |
22 |         s.spawn(|| counter.bump());
   |                 ^^
```

```
note: required by a bound in `Scope::<'scope, 'env>::spawn`
  --> /rustc/b940084d7eb6a299eb4bfeb8e34901bc051e7ac4/library/std/src/thread/scoped.rs:201:4
```

Within one thread, a sensor that counts its own reads can be shared freely:

```
/// A lab sensor that counts how often it has been read. `read` takes `&self`,
/// so many parts of a program can share the sensor, and still update the
/// counter.
struct Sensor {
    name: &'static str,
    reads: MyCell<u32>,
}

impl Sensor {
    fn read(&self) -> f64 {
        self.reads.set(self.reads.get() + 1);
        20.0 + f64::from(self.reads.get()) / 10.0
    }
}
```

```
let sensor = Sensor {
    name: "thermometer",
    reads: MyCell::new(0),
};
let first: &Sensor = &sensor; // two shared references to one sensor
let second: &Sensor = &sensor;
let r1 = first.read();
let r2 = second.read();
let r3 = first.read();
println!(
    "{}: {r1}, {r2}, {r3} after {} reads",
    sensor.name,
    sensor.reads.get()
);
```

```
thermometer: 20.1, 20.2, 20.3 after 3 reads
```

`Vec` and its invariant

A `Vec<T>` is a pointer, a capacity and a length, with the safety invariant above. Safe methods preserve it; unsafe ones let you take over:

```
/// Build `[0, 1, 4, 9, ...]` by writing into a vector's spare capacity, then
/// telling the vector how many elements are now initialized.
fn squares(n: usize) -> Vec<u64> {
    let mut v: Vec<u64> = Vec::with_capacity(n);
    let spare: &mut [MaybeUninit<u64>] = v.spare_capacity_mut();
    for (i, slot) in spare.iter_mut().take(n).enumerate() {
        slot.write((i as u64) * (i as u64));
    }
    // SAFETY: `set_len(n)` requires `n <= capacity` and that the elements
    // `0..n` are initialized. The capacity is at least `n` (`with_capacity`),
    // so `spare` has at least `n` slots, and the loop wrote all of the first
    // `n`. This re-establishes `Vec`'s invariant: the first `len` elements
    // are initialized and `len <= capacity`.
    unsafe { v.set_len(n) };
    v
}
```

```
squares(6) = [0, 1, 4, 9, 16, 25]
```

`spare_capacity_mut` hands out the uninitialized part of the buffer as `MaybeUninit<u64>` slots (a type whose validity invariant allows "uninitialized"); `set_len` declares them elements, and the `SAFETY:` comment proves that the invariant holds again.

32.5 Separation logic

The arguments of the last section all talk about *ownership* of memory, *disjointness*, and the memory an operation *does not touch*. Ordinary logic is clumsy with all three. A Hoare triple (precondition, command, postcondition) for a write,

$$\{\, x \mapsto 1 \land y \mapsto 1 \,\} \; * x = 3 \; \{\, x \mapsto 3 \land y \mapsto 1 \,\}$$

where $x \mapsto 1$ means "the cell at x holds 1", is false if x and y are the same address: the precondition holds, but afterwards the cell holds 3, not 1. So the precondition needs $x \neq y$, and with n pointers you need about $n^2/2$ such facts. **Separation logic**, introduced around 2000 by John Reynolds, Peter O'Hearn and their collaborators (Reynolds's 2002 paper is the standard reference), changes the meaning of assertions: an assertion describes a *piece* of the heap that you own, and the **separating conjunction** $P * Q$ says that the heap splits into two *disjoint* pieces satisfying P and Q. Then $x \mapsto 1 * y \mapsto 2$ implies $x \neq y$ by itself.

We build a *shallow* separation logic (assertions are Lean predicates, not a syntax of formulas) in `code/lean/Book/Ch32/SeparationLogic.lean`.

Heaps and assertions

```
/-- Memory locations. -/
abbrev Loc := ℕ
/-- The values stored in memory. -/
abbrev Val := ℤ

/-- A heap is a partial map from locations to values: `h l = none` means that
`l` is not allocated in `h`. -/
abbrev Heap := Loc → Option Val

/-- The empty heap. -/
def Heap.empty : Heap := fun _ => none

/-- The heap with exactly one allocated cell, `l`, holding `v`. -/
def Heap.single (l : Loc) (v : Val) : Heap :=
  fun l' => if l' = l then some v else none

/-- Two heaps are disjoint when no location is allocated in both. -/
def Heap.Disjoint (h₁ h₂ : Heap) : Prop := ∀ l, h₁ l = none ∨ h₂ l = none

/-- The union of two heaps (meant for disjoint heaps; if both define `l`, the
left one wins). -/
def Heap.union (h₁ h₂ : Heap) : Heap := fun l =>
  match h₁ l with
  | some v => some v
  | none => h₂ l

@[inherit_doc] scoped infixl:65 " ∪ₕ " => Heap.union
```

A heap is a partial map from locations to values (`abbrev` is like a Rust `type` alias). The union is meant for disjoint heaps; the `scoped` notation `∪ₕ` does not leak into other chapters. Real heaps are *finite*; the laws below do not need it, but finiteness is easy to state and is what allocation needs:

```
/-- A heap is finite when every location from some bound on is free. -/
def Heap.Finite (h : Heap) : Prop := ∃ n, ∀ l, n ≤ l → h l = none

/-- A finite heap always has a free location: this is what allocation needs. -/
theorem Heap.exists_free {h : Heap} (hf : h.Finite) : ∃ l, h l = none := by
  obtain ⟨n, hn⟩ := hf
  exact ⟨n, hn n (Nat.le_refl n)⟩
```

The bound `n` itself is a free location. (The file also proves that unions of finite heaps are finite. Mathlib's `Finmap` builds finiteness into the type.) Union is associative:

```
theorem union_assoc (h₁ h₂ h₃ : Heap) : (h₁ ∪ₕ h₂) ∪ₕ h₃ = h₁ ∪ₕ (h₂ ∪ₕ h₃) := by
  funext l
  simp only [union]
  cases h₁ l <;> rfl
```

`funext l` (function extensionality) reduces an equation between heaps to one location; `simp only [union]` unfolds the definition; `cases h₁ l` considers both possibilities for `h₁ l`, and in each both sides compute to the same thing (`<;>` applies `rfl` to every resulting goal). Commutativity needs disjointness, and disjointness of a union splits into disjointness of its parts; these are proved the same way:

```
/-- info: Ch32.Sep.Heap.union_comm {h₁ h₂ : Heap} (hd : h₁.Disjoint h₂) : h₁ ∪ₕ h₂ = h₂ ∪ₕ
h₁ -/
#guard_msgs in
#check union_comm

/--
info: Ch32.Sep.Heap.disjoint_union_left {h₁ h₂ h₃ : Heap} : (h₁ ∪ₕ h₂).Disjoint h₃ ↔
h₁.Disjoint h₃ ∧ h₂.Disjoint h₃
-/
#guard_msgs in
#check disjoint_union_left
```

Assertions are predicates on heaps:

```
/-- An assertion is a property of heaps: the heaps it describes. -/
abbrev Assertion := Heap → Prop

/-- `emp`: the heap is empty. -/
def emp : Assertion := fun h => h = Heap.empty

/-- `l ↦ v`: the heap is exactly one cell, `l`, holding `v`. -/
def pointsTo (l : Loc) (v : Val) : Assertion := fun h => h = Heap.single l v

/-- `P ∗ Q`: the heap splits into two disjoint parts, one satisfying `P` and
the other satisfying `Q`. -/
def sep (P Q : Assertion) : Assertion := fun h =>
  ∃ h₁ h₂, Heap.Disjoint h₁ h₂ ∧ h = h₁ ∪ₕ h₂ ∧ P h₁ ∧ Q h₂

/-- `P ⊢ Q`: every heap described by `P` is described by `Q`. -/
def Entails (P Q : Assertion) : Prop := ∀ h, P h → Q h

@[inherit_doc] scoped infix:60 " ↦ " => pointsTo
@[inherit_doc] scoped infixr:35 " ∗ " => sep
@[inherit_doc] scoped infix:25 " ⊢ " => Entails
```

`l ↦ v` holds only of the heap with exactly that one cell. $P * Q$ holds of a union of two disjoint heaps satisfying P and Q. Entailment $P \vdash Q$ is inclusion. (The turnstile here is entailment between assertions, as in the separation logic literature, not a typing judgment.)

The laws of the separating conjunction

Assertions are functions to `Prop`, so we state laws as equations: `funext h` reduces one to a single heap, and `propext` (propositional extensionality, the axiom you saw in Chapter 1's `#print axioms`) turns an equation between propositions into an "if and only if".

```
theorem sep_comm (P Q : Assertion) : (P ∗ Q) = (Q ∗ P) := by
  funext h
  apply propext
  constructor
  all_goals
    rintro ⟨h₁, h₂, hd, rfl, hP, hQ⟩
    exact ⟨h₂, h₁, hd.symm, union_comm hd, hQ, hP⟩
```

`all_goals` runs the same script on both directions. `rintro` takes apart a proof of $P * Q$ (two heaps, disjointness, the equation, which `rfl` substitutes away, and the two proofs), and `exact` rebuilds it with the heaps swapped, using `union_comm`, which needs exactly the disjointness we have.

```
theorem sep_assoc (P Q R : Assertion) : ((P ∗ Q) ∗ R) = (P ∗ (Q ∗ R)) := by
  funext h
  apply propext
  constructor
  · rintro ⟨h₁₂, h₃, hd, rfl, ⟨h₁, h₂, hd₁₂, rfl, hP, hQ⟩, hR⟩
    obtain ⟨hd₁₃, hd₂₃⟩ := disjoint_union_left.mp hd
    exact ⟨h₁, h₂ ∪ₕ h₃, disjoint_union_right.mpr ⟨hd₁₂, hd₁₃⟩,
      union_assoc h₁ h₂ h₃, hP, h₂, h₃, hd₂₃, rfl, hQ, hR⟩
  · rintro ⟨h₁, h₂₃, hd, rfl, hP, ⟨h₂, h₃, hd₂₃, rfl, hQ, hR⟩⟩
    obtain ⟨hd₁₂, hd₁₃⟩ := disjoint_union_right.mp hd
    exact ⟨h₁ ∪ₕ h₂, h₃, disjoint_union_left.mpr ⟨hd₁₃, hd₂₃⟩,
      (union_assoc h₁ h₂ h₃).symm, ⟨h₁, h₂, hd₁₂, rfl, hP, hQ⟩, hR⟩
```

Associativity re-splits $(h_1 \cup h_2) \cup h_3$ as $h_1 \cup (h_2 \cup h_3)$. The work is the disjointness bookkeeping: `disjoint_union_left` extracts from `hd` the facts "h_1 and h_3 are disjoint" and "h_2 and h_3 are disjoint", and `disjoint_union_right` combines what the new split needs. The other direction is the mirror image.

```
theorem emp_sep (P : Assertion) : (emp ∗ P) = P := by
  funext h
  apply propext
  constructor
  · rintro ⟨_, h₂, _, rfl, rfl, hP⟩
    exact hP
  · intro hP
    exact ⟨Heap.empty, h, fun _ => Or.inl rfl, rfl, rfl, hP⟩
```

From left to right, the `rfl` patterns force the first part to be the empty heap, so the whole heap is the second part; from right to left, we split `h` as the empty heap and `h` itself, which are disjoint because the empty heap has no cells. So assertions with $*$ and `emp` form a commutative monoid, like contexts with $\uplus$ in Chapter 21, which is no coincidence. One more law lets entailments act inside a separating conjunction:

```
/-- Monotonicity: entailments can be applied inside either side of `∗`.
Reading it with `Q = Q'`: what `P` entails, `P ∗ Q` entails with the frame `Q`
untouched. -/
theorem sep_mono {P P' Q Q' : Assertion} (hP : P ⊢ P') (hQ : Q ⊢ Q') :
    (P ∗ Q) ⊢ (P' ∗ Q') := by
  rintro h ⟨h₁, h₂, hd, rfl, hp, hq⟩
  exact ⟨h₁, h₂, hd, rfl, hP h₁ hp, hQ h₂ hq⟩
```

The same split works for both sides, with `hP` and `hQ` applied to the two parts. With $Q' = Q$ it is the logical core of the **frame** idea: what you conclude about your part of the heap still holds next to an untouched part Q, the *frame*.

Ownership is exclusive

$A \wedge A$ is the same as A, but $\ell \mapsto v * \ell \mapsto w$ is unsatisfiable:

```
/-- Nobody can own the same cell twice: `l ↦ v ∗ l ↦ w` describes no heap. -/
theorem pointsTo_sep_same (l : Loc) (v w : Val) :
    ((l ↦ v) ∗ (l ↦ w)) ⊢ fun _ => False := by
  rintro _ ⟨h₁, h₂, hd, _, rfl, rfl⟩
  rcases hd l with h | h <;> simp [Heap.single] at h
```

Both parts have the cell `l`, but disjointness says one of them does not. So contraction fails for points-to: $\ell \mapsto v \vdash \ell \mapsto v * \ell \mapsto v$ is false. This is E0499 one level down: you cannot own the same cell twice.

Arrays, and the lemma behind `split_at_mut`

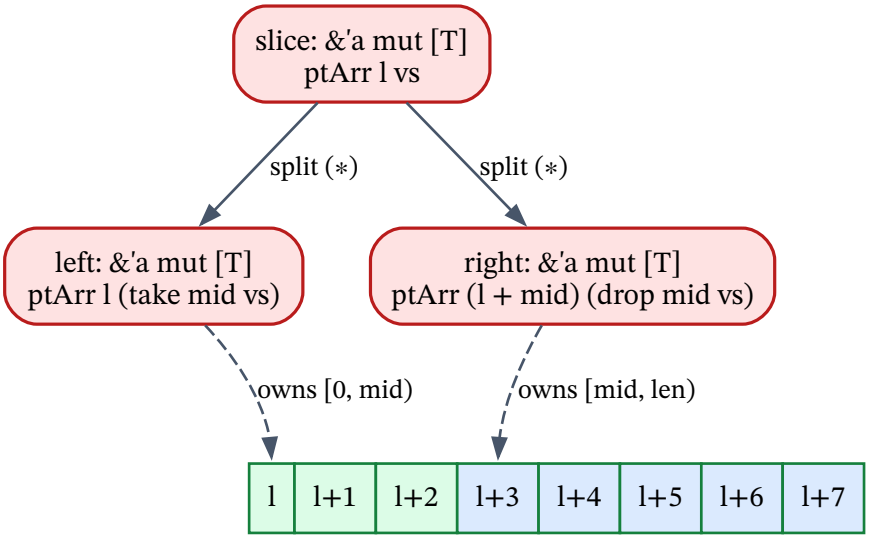
```
/-- `ptArr l [v₀, v₁, …]`: the cells `l, l+1, …` hold `v₀, v₁, …`. -/
def ptArr : Loc → List Val → Assertion
  | _, [] => emp
  | l, v :: vs => (l ↦ v) ∗ ptArr (l + 1) vs

/-- An array is the separating conjunction of its two parts. -/
theorem ptArr_append (xs ys : List Val) (l : Loc) :
    ptArr l (xs ++ ys) = (ptArr l xs ∗ ptArr (l + xs.length) ys) := by
  induction xs generalizing l with
  | nil => simp [ptArr, emp_sep]
  | cons x xs ih =>
    rw [List.cons_append, ptArr, ptArr, ih, sep_assoc, List.length_cons,
      Nat.add_assoc, Nat.add_comm 1 xs.length]

/-- `split_at_mut`, in the logic: owning `vs` at `l` is the same as owning its
first `mid` elements at `l` and the rest at `l + mid`. -/
theorem ptArr_split (vs : List Val) (l mid : ℕ) (h : mid ≤ vs.length) :
    ptArr l vs = (ptArr l (vs.take mid) ∗ ptArr (l + mid) (vs.drop mid)) := by
  have := ptArr_append (vs.take mid) (vs.drop mid) l
  rwa [List.take_append_drop, List.length_take_of_le h] at this
```

`ptArr_append` is proved by induction on `xs`, generalizing the start address (the hypothesis is needed at `l + 1`). In the `nil` case, `ptArr l []` is `emp` and `simp` finishes with `emp_sep`. In the `cons` case, the `rw` chain unfolds `ptArr`, applies the induction hypothesis, reassociates with `sep_assoc`, and fixes the address arithmetic `l + 1 + xs.length = l + (xs.length + 1)`. Then `ptArr_split` instantiates it with `vs.take mid` and `vs.drop mid`, using `List.take_append_drop` and `List.length_take_of_le`.

This is point 3 of the `SAFETY:` comment of `my_split_at_mut` as a theorem: owning a slice *is* owning its two halves separately. Splitting a `&mut [T]` is not contraction (nothing is duplicated); it is the division of one resource into two disjoint ones.



What split_at_mut does, in separation logic. The unique borrow of the whole slice owns cells l to l+7; splitting along the separating conjunction gives two unique borrows that own disjoint ranges of cells.

Hoare triples and the frame rule

`code/lean/Book/Ch32/Frame.lean` adds a tiny command language:

```
/-- Commands. -/
inductive Cmd where
  | skip
  | write (l : Loc) (v : Val)   -- `*l = v`
  | free (l : Loc)              -- deallocate `l`
  | swap (l₁ l₂ : Loc)          -- exchange the contents of `l₁` and `l₂`
  | seq (c₁ c₂ : Cmd)           -- `c₁; c₂`

/-- Change one cell of a heap (`none` deallocates it). -/
def Heap.update (h : Heap) (l : Loc) (o : Option Val) : Heap :=
  fun l' => if l' = l then o else h l'

/-- Run a command. `none` is a fault: the command touched a cell it does not
have. -/
def exec : Cmd → Heap → Option Heap
  | .skip, h => some h
  | .write l v, h => if (h l).isSome then some (h.update l (some v)) else none
  | .free l, h => if (h l).isSome then some (h.update l none) else none
  | .swap l₁ l₂, h =>
    match h l₁, h l₂ with
    | some a, some b => some ((h.update l₁ (some b)).update l₂ (some a))
    | _, _ => none
  | .seq c₁ c₂, h =>
    match exec c₁ h with
    | some h' => exec c₂ h'
    | none => none
```

A command faults (`none`) when it touches a cell that is not in its heap: a command may use only the cells it owns. A **Hoare triple** $\{P\}\ c\ \{Q\}$ says that from any heap satisfying P, c runs without fault to a heap satisfying Q:

```
/-- The Hoare triple `{P} c {Q}`: from every heap satisfying `P`, the command
runs without fault and ends in a heap satisfying `Q`. -/
def Triple (P : Assertion) (c : Cmd) (Q : Assertion) : Prop :=
  ∀ h, P h → ∃ h', exec c h = some h' ∧ Q h'
```

Specifications are **small**: they mention only the cells the command uses.

```
/-- Writing needs the cell, and only the cell. -/
theorem write_spec (l : Loc) (a v : Val) : Triple (l ↦ a) (.write l v) (l ↦ v) := by
  rintro _ rfl
  refine ⟨(Heap.single l a).update l (some v), by simp [exec, Heap.single], ?_⟩
  funext l'
  by_cases e : l' = l <;> simp [Heap.update, Heap.single, e]
```

The precondition makes the heap `Heap.single l a` (the `rfl` pattern); `refine` names the final heap and `simp` shows the write reaches it; then we compare the two heaps at every location, distinguishing `l' = l` from `l' ≠ l`. The specs of `free` and `swap` are proved alike:

```
/-- info: Ch32.Sep.free_spec (l : Loc) (a : Val) : Triple (l ↦ a) (Cmd.free l) emp -/
#guard_msgs in
#check free_spec

/--
info: Ch32.Sep.swap_spec (l₁ l₂ : Loc) (a b : Val) : Triple (l₁ ↦ a ∗ l₂ ↦ b) (Cmd.swap l₁
l₂) (l₁ ↦ b ∗ l₂ ↦ a)
-/
#guard_msgs in
#check swap_spec
```

The precondition of `swap` owns two cells *separately*. Rust's `std::mem::swap(x: &mut T, y: &mut T)` has this specification, and its signature guarantees the precondition: two live `&mut` are always disjoint. To use a small specification in a bigger heap we need the **frame rule**:

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame}$$

(With program variables there is a side condition that c does not modify the variables of R; our commands have none.) The rule is a theorem about the commands, resting on **locality**: a command that runs without fault in `h` runs the same way in `h ∪ₕ hf`, for any disjoint `hf`, and leaves `hf` alone. For a write, the key fact is that updating a cell the frame does not have commutes with adding the frame:

```
/-- Updating a cell that the frame does not have commutes with adding the
frame. -/
theorem update_union {h hf : Heap} {l : Loc} (o : Option Val) (hl : hf l = none) :
    (h ∪ₕ hf).update l o = h.update l o ∪ₕ hf := by
  funext l'
  by_cases e : l' = l
  · subst e
    cases o <;> simp [Heap.update, Heap.union, hl]
  · simp [Heap.update, Heap.union, e]
```

```
/-- Locality of `write`: if it runs without fault in `h`, it runs the same way
in `h` plus a disjoint frame `hf`, and leaves the frame alone. -/
theorem write_local {h h' hf : Heap} {l : Loc} {v : Val} (hd : Disjoint h hf)
    (he : exec (.write l v) h = some h') :
    Disjoint h' hf ∧ exec (.write l v) (h ∪ₕ hf) = some (h' ∪ₕ hf) := by
  simp only [exec] at he
  split at he
  · -- The write did not fault, so `l` is allocated in `h`, holding some `a`.
    rename_i hs
    obtain ⟨a, ha⟩ := Option.isSome_iff_exists.mp hs
    -- Then the frame does not have `l`.
    have hl := free_in_frame hd ha
    simp only [Option.some.injEq] at he
```

```
    subst he
    refine ⟨disjoint_update _ hd hl, ?_⟩
    simp [exec, Heap.union, ha, update_union _ hl]
  · -- The write faulted: `none = some h'` is impossible.
    simp at he
```

`simp only [exec] at he` unfolds the successful write into an `if` on whether `l` is allocated, and `split at he` considers both branches. In the first, `rename_i hs` names the hypothesis that `h l` is some value, and `Option.isSome_iff_exists` produces the value `a`. Since `h` has the cell, the disjoint frame does not (`free_in_frame`, a two-line lemma). Two facts remain: the updated heap is still disjoint from the frame (`disjoint_update`), and the write on the big heap gives the update plus the frame (`simp`, using `update_union`). In the second branch the write faulted, and `none = some h'` is absurd. `free` and `swap` are analogous, and locality of every command follows by recursion:

```
/-- Locality of every command, by recursion on the command. -/
theorem exec_frame : ∀ (c : Cmd) {h h' hf : Heap}, Disjoint h hf →
    exec c h = some h' → Disjoint h' hf ∧ exec c (h ∪ₕ hf) = some (h' ∪ₕ hf)
  | .skip, _, _, _, hd, he => by
    simp only [exec, Option.some.injEq] at he
    subst he
    exact ⟨hd, rfl⟩
  | .write _ _, _, _, _, hd, he => write_local hd he
  | .free _, _, _, _, hd, he => free_local hd he
  | .swap _ _, _, _, _, hd, he => swap_local hd he
  | .seq c₁ c₂, _, _, _, hd, he => by
    simp only [exec] at he
    split at he
    · -- `c₁` ran to some `hmid` (`he₁`), then `c₂` ran from `hmid` (`he`).
      rename_i hmid he₁
      obtain ⟨hd₁, he₁'⟩ := exec_frame c₁ hd he₁
      obtain ⟨hd₂, he₂'⟩ := exec_frame c₂ hd₁ he
      exact ⟨hd₂, by simp [exec, he₁', he₂']⟩
    · simp at he
```

For a sequence, `split at he` exposes the intermediate heap; locality is applied to `c₁` with the original frame, then to `c₂`, and the results combine. The frame rule follows:

```
/-- The frame rule: a triple about a small heap holds in any larger heap, with
the extra part `R` carried through unchanged. -/
theorem frame {P Q R : Assertion} {c : Cmd} (h : Triple P c Q) :
    Triple (P ∗ R) c (Q ∗ R) := by
  rintro _ ⟨h₁, h₂, hd, rfl, hp, hr⟩
  obtain ⟨h₁', he, hq⟩ := h h₁ hp
  obtain ⟨hd', he'⟩ := exec_frame c hd he
  exact ⟨h₁' ∪ₕ h₂, he', h₁', h₂, hd', rfl, hq, hr⟩
```

Split the heap as `h₁ ∪ₕ h₂`; the triple runs the command on `h₁` to some `h₁'` satisfying Q; locality says that on the whole heap it runs to `h₁' ∪ₕ h₂`, with the parts still disjoint: a split witnessing $Q * R$.

The frame rule makes separation logic *local*: each function is proved against the memory it uses, and the rule extends the result to any context. Rust's borrow checker gives safe code the same property statically: a function that receives `&mut a` cannot touch `b`.

32.6 RustBelt

We can now describe how the soundness of Rust was established. The RustBelt project (Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers and Derek Dreyer, "RustBelt: Securing the Foundations of

the Rust Programming Language", POPL 2018) followed the structure of this chapter's diagram, at full scale, with every proof checked in the Coq proof assistant (now called Rocq).

λRust. Rust has no formal semantics, so RustBelt defines a core language, λRust, close to MIR (Chapter 29): programs manipulate memory through explicit pointers, functions are in continuation-passing style, and types include owned pointers, shared and mutable references with lifetimes, products, sums and recursive types. Like every core calculus in this book, it is a model of Rust, not Rust. Its semantics makes use-after-free, out-of-bounds accesses and data races stuck.

Iris. Types are interpreted in Iris, a framework for *higher-order concurrent separation logic* developed by Jung, Krebbers, Birkedal, Dreyer and others. Iris adds to the separation logic above what a real language needs: *ghost state* (user-defined logical resources, such as the lifetime tokens below), *invariants* (facts about shared resources that every thread maintains), and *step-indexing* (a "later" modality $\triangleright$ that makes recursive definitions well founded).

Types as predicates, twice. In RustBelt each type has two interpretations. The *ownership predicate* says what it means for a thread to own a value of the type (for `Box<T>`, roughly: a pointer to memory holding an owned `T`, plus the right to free it). The *sharing predicate* says what it means for a location to be shared at the type for a lifetime κ: it interprets `&'κ T`. The sharing predicate must be *persistent*, Iris's word for freely duplicable, because shared references are `Copy`. For most types it means read-only access; for `Cell<T>` it is different, and that is where `Cell`'s soundness argument, and the reason it is not `Sync`, live.

The lifetime logic. References are interpreted with a logic of lifetimes built inside Iris. A lifetime κ is a *token*; a thread holds a fraction of it while κ is alive, and ending κ consumes the whole token and produces a *dead* token $[\dagger\ \kappa]$. A *full borrow* $\&^{\kappa}_{\text{full}}\ P$ is the right to access the resource P while κ is alive. Simplified (the real rules carry Iris's modalities), two rules say:

- **Borrowing.** Owning P can be exchanged for a full borrow of P at κ together with an *inheritance* that gives P back once $[\dagger\ \kappa]$ is available. This is what `let r = &mut x;` does: `r` gets the borrow, and `x` the inheritance.
- **Accessing.** A full borrow plus a fraction of κ's token can be exchanged for P, with the obligation to give P back to recover the token. This is dereferencing a mutable reference.

Lifetimes here are *resources* in a logic, and the rules of Chapter 28 (reborrowing, outlives, the escape check) become lemmas about them.

What was proved. The fundamental theorem for λRust's type system, adequacy (semantically well-typed programs do not get stuck), and semantic well-typedness of standard library abstractions that use unsafe code, among them `Cell`, `RefCell`, `Rc`, `Arc`, `Mutex`, `RwLock` and `thread::spawn`, each against its public signature. Along the way the authors found a real bug: `MutexGuard<T>` was declared `Sync` under a condition that was too weak. The result is our `link` theorem at the scale of a real language: a well-typed program that uses only verified libraries has no undefined behavior.

32.7 Aliasing models: Stacked Borrows, Tree Borrows and Miri

RustBelt assumes a semantics in which bad things get stuck. But which uses of raw pointers *are* bad? Consider:

```
let mut x = 0;
let p = &mut x as *mut i32;   // a raw pointer to x
let r1 = unsafe { &mut *p };  // a mutable reference made from p
let r2 = unsafe { &mut *p };  // a second one, made from p as well
*r1 += 1;
*r2 += 1;
```

No memory is freed or overrun, but `r1` and `r2` are two live mutable references to one place, which a `&mut` promises cannot happen, and the compiler optimizes on that promise (for instance, keeping `*r1` in a register across writes through other pointers). For such optimizations to be correct, some uses of raw pointers must be undefined behavior. An **aliasing model** is an operational semantics that says which.

Stacked Borrows (Ralf Jung, Hoang-Hai Dang, Jeehoon Kang and Derek Dreyer, POPL 2020) attaches to every location a stack of permissions, one per reference or raw pointer derived from it. Creating a reference pushes an item. Using a pointer requires its item to be in the stack, and writing through it pops every item above it (creating a mutable reference counts as such a use of the pointer it is made from). Using a pointer whose item was popped is undefined behavior. Above, creating `r2` from `p` pops `r1`'s item, so writing through `r1` is undefined behavior. **Tree Borrows**, a successor proposed by Neven Villani and collaborators, records a tree of which pointer was derived from which, and accepts patterns that real unsafe code uses and Stacked Borrows rejects. Neither is, at the time of writing, the official definition of undefined behavior in Rust; check the current status in the Rust project's Unsafe Code Guidelines.

Miri makes these models usable. It interprets MIR step by step and checks every step: in-bounds accesses, no use of freed or uninitialized memory, valid values for every type, the aliasing rules (Stacked Borrows, or Tree Borrows on request) and data races. It reports the first undefined behavior it finds. Miri is a component of the nightly toolchain:

```
rustup +nightly component add miri
cargo +nightly miri run --example ch32_unsafe_abstractions
MIRIFLAGS="-Zmiri-tree-borrows" cargo +nightly miri run --example ch32_unsafe_abstractions
cargo +nightly miri test -p <crate>
```

The book's scripts do not run Miri (it needs nightly and is slow), but it is a good habit for code with `unsafe`. It checks the executions you run, like a very strict test, so it finds nothing wrong with `bad_sub`. Miri finds bugs; only an argument for all inputs rules them out.

32.8 What types do not guarantee

The strongest claim of safe Rust is: no undefined behavior (memory safety, no data races), *provided* that the unsafe code it relies on is sound and the compiler is correct. Outside the guarantee:

- **panics**: out-of-bounds indexing, `unwrap` on `None`, integer overflow in debug builds (release builds wrap); a panic is a defined way to stop;
- **non-termination**: `loop {}` has every type (Chapter 10);
- **leaks**: `mem::forget` and `Rc` cycles are safe (Chapter 24);
- **deadlocks and race conditions**: data races are excluded, two threads waiting for each other's lock are not;
- **protocol completion and logic errors** (Chapter 25), and wrong answers;
- **unsound `unsafe` code and compiler bugs**: one unsound library voids the guarantee for every program that uses it, and rustc has known soundness bugs, tracked under the label `I-unsound` in the Rust issue tracker.

Each has an extension in the literature (totality checking, linear types, session types, refinement and dependent types), each with a cost. Lean, as Chapter 31 showed, can prove functional correctness, at the price of writing the proofs.

32.9 Where affinity reappears

A logic is *affine* (Chapter 21) when it has weakening but not contraction. We saw that contraction fails for points-to. Weakening, in separation logic, is $P * Q \vdash P$: you may forget a part of what you own. In our model it is false:

```
/-- In this exact model, the separating conjunction cannot forget a part: the
heap with two cells satisfies `0 ↦ 1 ∗ 1 ↦ 2` but not `0 ↦ 1`. -/
theorem weakening_fails :
    ¬ (((0 : Loc) ↦ 1) ∗ ((1 : Loc) ↦ 2) ⊢ ((0 : Loc) ↦ 1)) := by
  intro hent
  have hd : Disjoint (Heap.single 0 1) (Heap.single 1 2) := by
    intro l
    by_cases hl : l = 0 <;> simp [Heap.single, hl]
  have h := hent _ ⟨_, _, hd, rfl, rfl, rfl⟩
  -- `h` claims the two-cell heap equals the one-cell heap; look at cell 1.
  have := congrFun h 1
  simp [Heap.union, Heap.single] at this
```

The proof builds the two-cell heap, shows that it satisfies the left side (the single-cell heaps are disjoint, checked with `by_cases` on `l = 0`), applies the entailment, and gets an equation between the two-cell and the one-cell heap; `congrFun h 1` evaluates both at location 1, where they differ. The cause is the definition of points-to: `l ↦ v` holds of *exactly* one cell, so an assertion accounts for the whole heap. Such a logic is often called *linear* (or *classical*): a postcondition that omits a cell promises that it was freed.

The other natural model lets an assertion describe *at least* some cells:

```
/-- `h ≤ₕ h'`: `h'` extends `h` (it has every cell of `h`, with the same value,
and possibly more). -/
def Heap.Extends (h h' : Heap) : Prop := ∀ l v, h l = some v → h' l = some v

@[inherit_doc] scoped infix:50 " ≤ₕ " => Heap.Extends

/-- An assertion is up-closed (affine) when adding cells to a heap that
satisfies it gives a heap that still satisfies it. -/
def UpClosed (P : Assertion) : Prop := ∀ h h', h ≤ₕ h' → P h → P h'
```

```
theorem Heap.extends_union_left (h₁ h₂ : Heap) : h₁ ≤ₕ (h₁ ∪ₕ h₂) := by
  intro l v hv
  simp [Heap.union, hv]
```

```
/-- Weakening, `P ∗ Q ⊢ P`, holds for every up-closed `P`. -/
theorem sep_weaken {P Q : Assertion} (hP : UpClosed P) : (P ∗ Q) ⊢ P := by
  rintro _ ⟨h₁, h₂, _, rfl, hp, _⟩
  exact hP h₁ (h₁ ∪ₕ h₂) (Heap.extends_union_left h₁ h₂) hp
```

`extends_union_left` (proved by unfolding the union at a cell of `h₁`) says that `h₁` is extended by `h₁ ∪ₕ h₂`, and the up-closure of `P` carries the proof of `P` along. The points-to of this model has both properties of an affine resource:

```
/-- The affine points-to: the heap contains (at least) the cell `l` with `v`. -/
def pointsToA (l : Loc) (v : Val) : Assertion := fun h => h l = some v

theorem pointsToA_upClosed (l : Loc) (v : Val) : UpClosed (pointsToA l v) :=
  fun _ _ hext hp => hext l v hp

/-- Forgetting is allowed in the affine model. -/
example (l l' : Loc) (v v' : Val) :
    (pointsToA l v ∗ pointsToA l' v') ⊢ pointsToA l v :=
  sep_weaken (pointsToA_upClosed l v)

/-- Exclusive ownership survives: the same cell still cannot be owned twice. -/
theorem pointsToA_sep_same (l : Loc) (v w : Val) :
    (pointsToA l v ∗ pointsToA l w) ⊢ fun _ => False := by
  rintro _ ⟨h₁, h₂, hd, _, hv, hw⟩
  rcases hd l with h | h <;> simp_all [pointsToA]
```

It can be forgotten: it is up-closed, because an extension has every cell of the heap it extends, which is the definition of `≤ₕ`. And it cannot be duplicated: $*$ still demands disjoint parts, and the proof is the same case analysis as for `pointsTo_sep_same`. Finally, the up-closed assertions are closed under $*$, so they form a logic of their own; the proof gives the left part to P unchanged and everything else to Q:

```
/-- info: Ch32.Sep.upClosed_sep {P Q : Assertion} (hQ : UpClosed Q) : UpClosed (P ∗ Q) -/
#guard_msgs in
#check upClosed_sep
```

Iris is an affine separation logic: $P * Q \vdash P$ holds for every assertion. Its propositions are resources that may always be forgotten but in general not duplicated; the ones that may be are called *persistent* (the analogue of $!A$ in Chapter 23, and of `Copy`). This fits Rust: a logic in which resources could not be forgotten would be the wrong tool for a language in which `mem::forget` is safe and a leaked `Box` is not undefined behavior. Leaking is weakening.

So "affine" describes something in the foundations of Rust with full precision: the *logic* in which Rust's soundness was proved. But look at what its affine resources are: memory cells, lifetime tokens, full borrows, the ghost resources from which the *meanings* of Rust's types are built, not the values of the affine λ-calculus of Chapter 22. Owning a `Box<T>` and holding a mutable borrow are affine resources of the logic; a shared reference is a persistent (duplicable) predicate; lifetimes are fractional tokens. The affine structure sits one level below the type system, in the model, while the surface discipline for borrows and lifetimes is the region-and-permission system of Part VI. Blurring the two levels is

an easy route to the folklore claim of Chapter 1, and Chapter 33 keeps them apart when it gives the verdict.

32.10 Putting it together: from `split_at_mut` to the frame rule

Let us follow one small program through the levels of this chapter:

```
let mut cells = [10, 20, 30];
let (front, back) = my_split_at_mut(&mut cells, 2);
front.swap(0, 1); // touches cells 0 and 1 only
back[0] = 0; // touches cell 2 only
println!("cells = {cells:?}");
```

```
cells = [20, 10, 0]
```

The type checker. rustc checks `main` against the *signature* of `my_split_at_mut`: two `&mut [i32]` with one lifetime, held in two variables, so `front` and `back` may be used side by side. This is the client's syntactic typing.

The library's obligation. The `SAFETY:` comment of `my_split_at_mut` is an informal proof that it is *semantically* well typed: for every slice and `mid`, it panics or returns two unique borrows of disjoint memory.

The logic. The ownership of `cells` at address ℓ is $\mathit{ptArr}\ \ell\ [10, 20, 30]$, which `ptArr_split` with $mid = 2$ turns into $\mathit{ptArr}\ \ell\ [10, 20] * \mathit{ptArr}\ (\ell + 2)\ [30]$, one resource for `front` and one for `back`. Each statement works on its own resource, and the frame rule says the other is untouched. In Lean, with the cells at locations 0, 1 and 2:

```
/-- Three cells; swap the first two, then clear the third. Each step is proved
with its small specification, and the frame rule supplies the cells that the
step does not touch. -/
theorem three_cells :
    Triple ((0 ↦ 10) ∗ (1 ↦ 20) ∗ (2 ↦ 30))
      (.seq (.swap 0 1) (.write 2 0))
      ((0 ↦ 20) ∗ (1 ↦ 10) ∗ (2 ↦ 0)) := by
  apply seq_rule (Q := (0 ↦ 20) ∗ (1 ↦ 10) ∗ (2 ↦ 30))
  · -- The swap owns cells 0 and 1; cell 2 is the frame.
    have h := frame (R := 2 ↦ 30) (swap_spec 0 1 10 20)
    rw [sep_assoc, sep_assoc] at h
    exact h
  · -- The write owns cell 2; cells 0 and 1 are the frame.
    have h := frame (R := (0 ↦ 20) ∗ (1 ↦ 10)) (write_spec 2 30 0)
    rw [sep_comm (2 ↦ 30), sep_comm (2 ↦ 0), sep_assoc, sep_assoc] at h
    exact h
```

The sequencing rule (proved in the file like the frame rule) needs an intermediate assertion `Q`, the state after the swap. For the swap, the small axiom `swap_spec 0 1 10 20` mentions only cells 0 and 1; `frame` adds cell 2, nested as $((0 \mapsto 10) * (1 \mapsto 20)) * (2 \mapsto 30)$, and two rewrites with `sep_assoc` produce the right-nested form of the goal. For the write, `write_spec 2 30 0` mentions only cell 2; `frame` puts cells 0 and 1 on the right, so `sep_comm` moves cell 2 to the right before reassociating. Each step was proved against the smallest heap it needs, and the frame rule did the rest: the same reason that lets rustc check `front.swap(0, 1)` without thinking about `back`.

32.11 Common confusions

"**`unsafe` turns off the borrow checker.**" It does not: all the usual checks apply inside an `unsafe` block, which only permits five extra operations.

"**If my tests pass and Miri is happy, my unsafe code is sound.**" Soundness of a safe API is a claim about *all* callers and inputs; tests and Miri examine the executions you run. `bad_sub` passes both.

"**A sound function never panics.**" `my_split_at_mut` panics when `mid > len`, and that is what makes it sound. A panic is defined behavior; reading past the end of a slice is not.

"**$P * Q$ is just $P \wedge Q$.**" For resources they differ completely: $\ell \mapsto v \wedge \ell \mapsto v$ is $\ell \mapsto v$, while $\ell \mapsto v * \ell \mapsto v$ is impossible.

"**Iris is affine, so Rust's type system is affine.**" Affinity is a property of the logic in which the model is built. The surface rules for borrows, with duplicable shared references and lifetimes, are a different system, related to the model by a theorem, not by identity.

32.12 Summary

- Syntactic soundness says nothing about code without a derivation, and safe Rust depends on such code: the standard library is built on `unsafe`.
- Semantic typing interprets types as predicates on values; at function type, "good arguments go to good results", which makes it a *logical* relation.
- The fundamental theorem ($\vdash$ implies $\models$) is proved with one compatibility lemma per rule. In Lean we proved it for a language with an unchecked primitive, verified `satSub`, refuted `badSub`, and linked verified libraries with type-checked clients.
- An `unsafe` block discharges proof obligations, an `unsafe fn` passes them to its caller, and a `SAFETY:` comment is the proof. Safety invariants are the semantic types; validity invariants are what the compiler relies on.
- Separation logic describes owned pieces of the heap: $*$ is a commutative monoid with unit `emp`, points-to cannot be duplicated, arrays split along $*$ (the lemma behind `split_at_mut`), and the frame rule follows from locality.
- RustBelt proved Rust's type system sound in this style, with λRust, Iris and a lifetime logic. Stacked and Tree Borrows define what unsafe code may do with aliases; Miri checks the executions you run.
- Iris is affine ($P * Q \vdash P$): affinity reappears one level below Rust's type system, in the logic used to prove it sound.

32.13 Exercises

1. **A second unchecked primitive.** Extend `Language.lean` with `predRaw e`, an unchecked predecessor that is stuck on 0 (a constructor, an `Eval` rule with premise `0 < n`, a case in `run`). Define `safePred := fun x => if x ≤ 0 then 0 else predRaw x` and prove

`SemTyped [] safePred (.arr .nat .nat)`. Which proofs had to change, and why did `fundamental` not need a new case? *Hint:* follow `satSub_sem`. `HasType` has no rule for the primitive, so `induction h` produces the same cases as before; only `run_sound` grows.

2. **Contraction fails, weakening fails.** Prove `sep_emp : (P ∗ emp) = P` from `sep_comm` and `emp_sep`, and prove `¬ (((0 : Loc) ↦ 1) ⊢ ((0 : Loc) ↦ 1) ∗ ((0 : Loc) ↦ 1))`. Which structural rule of Chapter 21 does this fact correspond to, and which does `weakening_fails`? *Hint:* apply the supposed entailment to `Heap.single 0 1` and feed the result to `pointsTo_sep_same`.
3. **A non-local command.** Add to `Frame.lean` a command `checkEmpty` that succeeds (leaving the heap unchanged) only on the empty heap. Show that `Triple emp .checkEmpty emp` holds but the framed triple with `R := (0 : Loc) ↦ 1` does not. Which step of `exec_frame` fails? *Hint:* the command inspects cells it does not own. In Rust the analogue is a function whose behavior depends on memory it has no reference to.
4. **Safe code on top of `my_split_at_mut`.** Using `my_split_at_mut` and no `unsafe` of your own, write `fn swap_ends<T>(v: &mut [T])` (swap the first and last elements of a slice of length at least 2) and `fn first_and_rest<T>(v: &mut [T]) -> Option<(&mut T, &mut [T])>`. Why do they need no `SAFETY:` comments? *Hint:* split at 1 or at `len - 1`. Your functions are clients checked by the rules, like `client` in `LogicalRelation.lean`.
5. **Why `Cell::get` needs `Copy`.** Suppose `MyCell` had `fn get_clone(&self) -> T where T: Clone`, calling `(*self.value.get()).clone()`. Design a type whose `Clone` makes this unsound and describe the execution that goes wrong (if you run it, use Miri). *Hint:* give the type an `Rc` to a structure containing the cell, and let `clone` call `set` on it while `get_clone` holds a reference to the old contents.
6. **Two kinds of invariant.** Classify as validity or safety invariants: a `bool` is 0 or 1; a `Vec` has `len <= capacity`; a `&T` is non-null; a `str` is UTF-8; a `NonZeroU8` is not 0; a `SortedVec` newtype (Chapter 30's "parse, don't validate") is sorted. *Hint:* does the compiler use the fact, for layout or optimization, or only the library's code? `size_of::<Option<T>>()` is a useful experiment.